

# BLG 475E – Software Quality and Testing

## Term Project Report:

### LLM-Based Test Generation and Evaluation Using the HumanEval Dataset

**Ahmet Enes Çiğdem**

150220079

**Ali Eren Çiftçi**

150220022

**Taha Çali**

150220050

May 23, 2026

#### Abstract

In this report, we go over the work we did for the BLG 475E Software Quality and Testing term project. We picked two Large Language Models—Gemini Pro (LLM 1) and Claude Opus 4.6 (LLM 2)—and tested how well they can write code and generate unit tests for 30 Java tasks from the HumanEval dataset. Since API costs were too high, we used a semi-agentic workflow where we gave prompts through each model’s web interface and then put the generated code into our Maven project by hand. Our testing pipeline had three stages: first we ran the base tests and checked branch coverage with JaCoCo, then we asked the LLMs to write better tests to cover missed branches and we ran JNose for test smells, and finally we wrote manual black-box tests using equivalence class partitioning and boundary value analysis. Both LLMs got to 100% branch coverage after the improvement step. But the manual tests still found 2 bugs in LLM 1’s code and 4 bugs in LLM 2’s code—things like integer overflow, double overflow, missing input checks, and slow algorithms. We fixed all the bugs with help from the LLMs. The main takeaway is that even with full coverage, you still need manual boundary testing to catch edge-case bugs.

# Contents

|          |                                                                                                     |           |
|----------|-----------------------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction and Literature Review</b>                                                           | <b>3</b>  |
| 1.1      | Introduction . . . . .                                                                              | 3         |
| 1.2      | Literature Review . . . . .                                                                         | 3         |
| 1.2.1    | ChatUnitTest: A Framework for LLM-Based Test Generation . . . . .                                   | 3         |
| 1.2.2    | Mutation-Guided LLM-Based Test Generation at Meta . . . . .                                         | 3         |
| 1.2.3    | Adaptive Testing for LLM-Based Applications . . . . .                                               | 4         |
| 1.2.4    | LLM-Based Code Generation for Golang Compiler Testing . . . . .                                     | 4         |
| 1.2.5    | LLM-Based Test-Driven Interactive Code Generation . . . . .                                         | 4         |
| <b>2</b> | <b>LLM Selection and Methodology</b>                                                                | <b>4</b>  |
| 2.1      | Selected LLMs . . . . .                                                                             | 4         |
| 2.2      | Semi-Agentive Approach Justification . . . . .                                                      | 5         |
| 2.3      | Advantages and Limitations . . . . .                                                                | 5         |
| <b>3</b> | <b>Phase 1: Selecting LLMs, LLM-based code and test generation, and manual unit test generation</b> | <b>5</b>  |
| 3.1      | Prompt Selection Criteria . . . . .                                                                 | 5         |
| 3.2      | Difficulty Categorization . . . . .                                                                 | 6         |
| 3.3      | Base Testing Phase . . . . .                                                                        | 6         |
| 3.4      | Test Smell and Branch Coverage Analysis . . . . .                                                   | 7         |
| 3.5      | Manual Assessment Results . . . . .                                                                 | 10        |
| 3.6      | Coverage Discussion and Input Mutations . . . . .                                                   | 11        |
| <b>4</b> | <b>Results, Analysis, and Refactoring</b>                                                           | <b>11</b> |
| 4.1      | Test Case Results . . . . .                                                                         | 11        |
| 4.2      | Statistical Evaluation . . . . .                                                                    | 12        |
| 4.3      | LLM Comparison . . . . .                                                                            | 13        |
| 4.4      | Refactoring Process . . . . .                                                                       | 14        |
| 4.5      | Agent-Generated Code Errors . . . . .                                                               | 14        |
| <b>5</b> | <b>Conclusion</b>                                                                                   | <b>15</b> |
| <b>6</b> | <b>Acknowledgments</b>                                                                              | <b>16</b> |
| 6.1      | GitHub Repository . . . . .                                                                         | 16        |
| 6.2      | Work Distribution . . . . .                                                                         | 16        |

# 1 Introduction and Literature Review

## 1.1 Introduction

Large Language Models (LLMs) like Google’s Gemini and Anthropic’s Claude have become popular tools in software development. They can write code from plain English descriptions and they can also generate unit tests, which makes them useful for speeding up the development process.

Writing tests by hand takes a lot of time and effort, and it is easy to miss edge cases. LLMs can help with this by generating test cases from function signatures and docstrings. But the code that LLMs write is not always correct—research has shown that it can have subtle bugs, especially around boundary conditions and performance [1, 2].

In this project, we used two LLMs—Gemini Pro and Claude Opus 4.6—to generate Java solutions and tests for 30 tasks from the HumanEval benchmark [6]. We then checked the quality of both the code and the tests through a multi-step process: base testing, coverage-driven test improvement, test smell detection, manual black-box testing with equivalence classes and boundary values, and finally LLM-assisted bug fixing.

## 1.2 Literature Review

This section reviews five recent studies that investigate the use of LLMs in automated test generation and code quality assurance.

### 1.2.1 ChatUniTest: A Framework for LLM-Based Test Generation

Chen et al. [1] present *ChatUniTest*, an automated unit test generation framework that addresses two key weaknesses of LLM-based tools: limited context windows and insufficient validation of generated tests. The framework introduces an *adaptive focal context* mechanism that extracts only the relevant method, class properties, and dependencies via AST parsing, rather than feeding the entire source file to the model. Generated tests pass through a Generation-Validation-Repair cycle that includes syntactic, compilation, and runtime checks, with both rule-based and LLM-based repair strategies. In evaluation, ChatUniTest achieved 59.6% line coverage, outperforming TestSpark (42.1%) and EvoSuite (38.2%). This paper is directly relevant to our project since our workflow similarly relies on iterative prompt construction and post-generation validation to obtain compilable and correct unit tests from LLMs.

### 1.2.2 Mutation-Guided LLM-Based Test Generation at Meta

Foster et al. [2] describe Meta’s *Automated Compliance Hardening (ACH)* system, which combines mutation testing with LLM-based test generation. Rather than prompting an LLM to “write tests” generically, ACH first generates targeted mutants—simulated faults specific to a concern such as privacy—and then instructs the LLM to produce tests that kill these mutants. Deployed on 10,795 Kotlin classes across seven Meta platforms, ACH generated 571 hardening test cases with a 73% developer acceptance rate. A notable finding is that 49% of the valuable tests did not increase line coverage, demonstrating that coverage alone is an insufficient quality metric. This insight directly informs our manual assessment phase, where we complement coverage analysis with equivalence class partitioning and boundary testing.

### 1.2.3 Adaptive Testing for LLM-Based Applications

Yoon et al. [3] address the challenge of efficiently testing LLM-powered applications by adapting *Adaptive Random Testing (ART)* to text-based prompt templates. Their approach iteratively selects the most diverse candidate input—measured via string distance metrics such as Levenshtein and Jaccard—and dynamically adjusts selection based on prior pass/fail outcomes. The technique discovers failures significantly faster than random sampling while requiring fewer costly API calls. For our project, this work provides a theoretical foundation for the input diversity strategies we apply when designing equivalence classes and selecting varied test inputs from the HumanEval dataset.

### 1.2.4 LLM-Based Code Generation for Golang Compiler Testing

Gu et al. [4] investigate using LLMs as intelligent fuzzers for compiler testing. By fine-tuning an LLM on a carefully filtered Go codebase and guiding generation with a coverage-based seed scheduling strategy, the authors achieved 3.38% compiler code coverage—nearly eight times higher than standard Go fuzzing (0.44%). The generated code exhibited only 2.79% syntax errors and effectively zero undefined behaviors. Although our project targets Java unit tests rather than compiler fuzzing, this paper demonstrates how constrained, seed-guided LLM generation can produce high-quality code artifacts, reinforcing the value of prompt engineering and structured generation strategies in our own pipeline.

### 1.2.5 LLM-Based Test-Driven Interactive Code Generation

Fakhoury et al. [5] propose *TiCoder*, an interactive workflow where the LLM first generates unit tests from a natural language prompt to formalize user intent, and then produces the implementation code constrained by those tests. In a user study with 15 programmers, participants using TiCoder were significantly more likely to correctly evaluate AI-generated code and reported lower cognitive load. At scale, the approach improved pass@1 accuracy by approximately 46% within five interaction rounds. This paper is highly relevant to our project: it validates the test-driven philosophy we adopt in Phase 1, where LLM-generated tests serve as the specification against which code correctness is evaluated, and it motivates the iterative refinement approach we use in both our agentic and semi-agentic workflows.

## 2 LLM Selection and Methodology

### 2.1 Selected LLMs

We chose two different LLMs for this project. Both are known for being good at writing code and reasoning about problems.

Table 1: Comparison of Selected LLMs

| Criterion     | LLM 1                        | LLM 2                        |
|---------------|------------------------------|------------------------------|
| Model Name    | Gemini Pro                   | Claude Opus 4.6              |
| Access Method | Web Interface (Semi-Agentic) | Web Interface (Semi-Agentic) |
| Primary Role  | Code and test generation     | Code and test generation     |

## 2.2 Semi-Agentic Approach Justification

At first, we tried to build a fully automated pipeline using Python scripts that would call the models' APIs directly. But we ran into problems with API rate limits and token quotas—the free tiers ran out fast when dealing with 30 tasks, and paying for more was not an option.

So we switched to a *semi-agentic approach* instead. We wrote the prompts on our machines, pasted them into each LLM's web interface, and then copied the generated code back into our Maven project by hand.

## 2.3 Advantages and Limitations

This approach had some clear pros and cons.

### Advantages:

- *No cost:* Using the web interface meant we did not have to pay anything or worry about token limits.
- *More control:* We could see exactly what the model was doing at each step and fix our prompts if something went wrong.

### Limitations:

- *Lots of manual work:* Copying and pasting code for 30 tasks across two models took a long time.
- *Not scalable:* This would not work well for the full HumanEval dataset (164 tasks) or for a CI/CD pipeline.

# 3 Phase 1: Selecting LLMs, LLM-based code and test generation, and manual unit test generation

## 3.1 Prompt Selection Criteria

We picked 30 prompts from the HumanEval dataset [6]. The full list can be found in `dataset/selected_prompts.json`. Our selection criteria were:

- **Difficulty mix:** 15 easy, 10 moderate, and 5 hard prompts so we could test the LLMs at different complexity levels.
- **Different topics:** We made sure to include string operations, math problems, list processing, and algorithm tasks.

- **Required tasks:** Tasks #18, #23, and #27 were mandatory for Phase 1.2.
- **How we classified difficulty:** Easy tasks have simple logic and few edge cases. Moderate tasks need multiple steps or careful handling of special inputs. Hard tasks involve recursion, complex math, or tricky algorithms.

### 3.2 Difficulty Categorization

Table 2 summarizes the categorization of the 30 selected prompts by difficulty level.

Table 2: HumanEval Prompt Categorization by Difficulty

| Difficulty    | Task IDs                                                              | Rationale                  |
|---------------|-----------------------------------------------------------------------|----------------------------|
| Easy (15)     | #2, #5, #8, #9, #14, #15, #16, #23, #27, #28, #30, #34, #35, #42, #45 | Simple logic, few branches |
| Moderate (10) | #0, #3, #7, #11, #13, #18, #26, #29, #31, #43                         | Multi-step, edge cases     |
| Hard (5)      | #4, #10, #25, #39, #49                                                | Recursion, complex math    |

### 3.3 Base Testing Phase

We gave each LLM the HumanEval prompt—function signature, docstring, and example test cases—and asked it to write the Java implementation. The code went into `src/main/java/humaneval/llm1/` and `src/main/java/humaneval/llm2/`.

At the same time, each LLM also generated a basic set of unit tests (the “base tests”) based on the examples in the prompt. These tests were simple and just checked that the function works for the sample inputs. We did not change the LLM-generated source code at this point.

Figure 1 and Figure 2 show the JaCoCo coverage results for these base tests. LLM 1 had mixed results, with branch coverage ranging from 68% to 100%—for example, Task #18 only got 68% and Task #7 got 70%. LLM 2 did better overall, with only Task #25 falling below 100% (at 83%). Some simple tasks like `strlen` show “n/a” because they have no branches to cover.

|                       |  |      |  |      |   |    |   |    |   |   |   |   |
|-----------------------|--|------|--|------|---|----|---|----|---|---|---|---|
| humaneval.llm1.task11 |  | %86  |  | %75  | 2 | 6  | 2 | 20 | 0 | 2 | 0 | 1 |
| humaneval.llm1.task18 |  | %91  |  | %68  | 5 | 10 | 2 | 15 | 0 | 2 | 0 | 1 |
| humaneval.llm1.task15 |  | %93  |  | %83  | 1 | 5  | 1 | 9  | 0 | 2 | 0 | 1 |
| humaneval.llm1.task7  |  | %94  |  | %70  | 3 | 7  | 1 | 9  | 0 | 2 | 0 | 1 |
| humaneval.llm1.task0  |  | %96  |  | %80  | 2 | 7  | 1 | 9  | 0 | 2 | 0 | 1 |
| humaneval.llm1.task31 |  | %96  |  | %85  | 2 | 9  | 1 | 11 | 0 | 2 | 0 | 1 |
| humaneval.llm1.task4  |  | %96  |  | %75  | 2 | 6  | 1 | 13 | 0 | 2 | 0 | 1 |
| humaneval.llm1.task39 |  | %97  |  | %94  | 1 | 12 | 0 | 18 | 0 | 3 | 0 | 1 |
| humaneval.llm1.task23 |  | %100 |  | n/a  | 0 | 2  | 0 | 2  | 0 | 2 | 0 | 1 |
| humaneval.llm1.task35 |  | %100 |  | n/a  | 0 | 2  | 0 | 2  | 0 | 2 | 0 | 1 |
| humaneval.llm1.task45 |  | %100 |  | n/a  | 0 | 2  | 0 | 2  | 0 | 2 | 0 | 1 |
| humaneval.llm1.task2  |  | %100 |  | n/a  | 0 | 2  | 0 | 3  | 0 | 2 | 0 | 1 |
| humaneval.llm1.task34 |  | %100 |  | n/a  | 0 | 2  | 0 | 2  | 0 | 2 | 0 | 1 |
| humaneval.llm1.task42 |  | %100 |  | n/a  | 0 | 3  | 0 | 4  | 0 | 3 | 0 | 1 |
| humaneval.llm1.task30 |  | %100 |  | %100 | 0 | 4  | 0 | 4  | 0 | 3 | 0 | 1 |
| humaneval.llm1.task13 |  | %100 |  | %100 | 0 | 3  | 0 | 9  | 0 | 2 | 0 | 1 |
| humaneval.llm1.task29 |  | %100 |  | %75  | 1 | 5  | 0 | 4  | 0 | 3 | 0 | 1 |
| humaneval.llm1.task3  |  | %100 |  | %100 | 0 | 4  | 0 | 8  | 0 | 2 | 0 | 1 |
| humaneval.llm1.task14 |  | %100 |  | %83  | 1 | 5  | 0 | 7  | 0 | 2 | 0 | 1 |
| humaneval.llm1.task28 |  | %100 |  | %83  | 1 | 5  | 0 | 8  | 0 | 2 | 0 | 1 |
| humaneval.llm1.task33 |  | %100 |  | %100 | 0 | 4  | 0 | 8  | 0 | 2 | 0 | 1 |
| humaneval.llm1.task9  |  | %100 |  | %100 | 0 | 5  | 0 | 11 | 0 | 2 | 0 | 1 |
| humaneval.llm1.task16 |  | %100 |  | %83  | 1 | 5  | 0 | 7  | 0 | 2 | 0 | 1 |
| humaneval.llm1.task26 |  | %100 |  | %83  | 1 | 6  | 0 | 8  | 0 | 3 | 0 | 1 |
| humaneval.llm1.task5  |  | %100 |  | %87  | 1 | 6  | 0 | 9  | 0 | 2 | 0 | 1 |
| humaneval.llm1.task8  |  | %100 |  | %100 | 0 | 3  | 0 | 11 | 0 | 2 | 0 | 1 |
| humaneval.llm1.task49 |  | %100 |  | %100 | 0 | 5  | 0 | 12 | 0 | 2 | 0 | 1 |
| humaneval.llm1.task25 |  | %100 |  | %100 | 0 | 6  | 0 | 12 | 0 | 2 | 0 | 1 |
| humaneval.llm1.task27 |  | %100 |  | %90  | 1 | 7  | 0 | 11 | 0 | 2 | 0 | 1 |
| humaneval.llm1.task10 |  | %100 |  | %100 | 0 | 7  | 0 | 19 | 0 | 3 | 0 | 1 |

Figure 1: JaCoCo Branch Coverage — LLM 1 Base Tests

|                       |  |      |  |      |   |   |   |    |   |   |   |   |
|-----------------------|--|------|--|------|---|---|---|----|---|---|---|---|
| humaneval.llm2.task23 |  | %100 |  | n/a  | 0 | 2 | 0 | 2  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task28 |  | %100 |  | n/a  | 0 | 2 | 0 | 2  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task2  |  | %100 |  | n/a  | 0 | 2 | 0 | 2  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task35 |  | %100 |  | n/a  | 0 | 2 | 0 | 2  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task45 |  | %100 |  | n/a  | 0 | 2 | 0 | 2  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task34 |  | %100 |  | n/a  | 0 | 2 | 0 | 4  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task29 |  | %100 |  | n/a  | 0 | 3 | 0 | 2  | 0 | 3 | 0 | 1 |
| humaneval.llm2.task42 |  | %100 |  | n/a  | 0 | 3 | 0 | 2  | 0 | 3 | 0 | 1 |
| humaneval.llm2.task30 |  | %100 |  | %100 | 0 | 4 | 0 | 2  | 0 | 3 | 0 | 1 |
| humaneval.llm2.task13 |  | %100 |  | %100 | 0 | 3 | 0 | 6  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task49 |  | %100 |  | %100 | 0 | 3 | 0 | 5  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task14 |  | %100 |  | %100 | 0 | 3 | 0 | 5  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task15 |  | %100 |  | %100 | 0 | 4 | 0 | 7  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task3  |  | %100 |  | %100 | 0 | 4 | 0 | 8  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task31 |  | %100 |  | %100 | 0 | 5 | 0 | 7  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task7  |  | %100 |  | %100 | 0 | 4 | 0 | 7  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task18 |  | %100 |  | %100 | 0 | 4 | 0 | 6  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task16 |  | %100 |  | %100 | 0 | 3 | 0 | 5  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task11 |  | %100 |  | %100 | 0 | 4 | 0 | 7  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task27 |  | %100 |  | %100 | 0 | 4 | 0 | 8  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task43 |  | %100 |  | %100 | 0 | 5 | 0 | 6  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task25 |  | %100 |  | %83  | 1 | 5 | 0 | 11 | 0 | 2 | 0 | 1 |
| humaneval.llm2.task8  |  | %100 |  | %100 | 0 | 3 | 0 | 8  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task0  |  | %100 |  | %100 | 0 | 5 | 0 | 7  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task5  |  | %100 |  | %100 | 0 | 4 | 0 | 9  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task9  |  | %100 |  | %100 | 0 | 5 | 0 | 11 | 0 | 2 | 0 | 1 |
| humaneval.llm2.task39 |  | %100 |  | %100 | 0 | 8 | 0 | 17 | 0 | 3 | 0 | 1 |
| humaneval.llm2.task10 |  | %100 |  | %100 | 0 | 7 | 0 | 15 | 0 | 3 | 0 | 1 |
| humaneval.llm2.task4  |  | %100 |  | %100 | 0 | 4 | 0 | 11 | 0 | 2 | 0 | 1 |
| humaneval.llm2.task26 |  | %100 |  | %100 | 0 | 5 | 0 | 6  | 0 | 3 | 0 | 1 |

Figure 2: JaCoCo Branch Coverage — LLM 2 Base Tests

### 3.4 Test Smell and Branch Coverage Analysis

We ran test smell analysis using JNose on all our test classes (base, improved, and manual). The raw results are saved in `analysis/test_smell_reports/result_byclasstest.csv`.

Table 3 shows what we found.

Table 3: Test Smell Analysis Summary

| Category        | Smell Type        | Description                                        | Prevalence       |
|-----------------|-------------------|----------------------------------------------------|------------------|
| Base Tests      | Unknown Test      | Test methods have no clear name or assertion       | All 60 classes   |
| Base Tests      | Ignored Test      | Test annotation is there but the method is skipped | All 60 classes   |
| Improved (LLM1) | Magic Number Test | Hard-coded numbers in assertions                   | 20 of 30 classes |
| Improved (LLM2) | Magic Number Test | Hard-coded numbers in assertions                   | 18 of 30 classes |
| Manual Tests    | Magic Number Test | Numbers used in boundary value tests               | 14 of 20 classes |
| Manual Tests    | Verbose Test      | Task #10 has 17 methods and 143 lines              | 1 class          |

The main smells in the base tests were *Unknown Test* and *Ignored Test*. This makes sense because the LLMs tended to put everything into one big test method without giving it a good name. In the improved and manual tests, *Magic Number Test* came up a lot, but that is hard to avoid when you are writing boundary value tests with specific numbers.

After looking at which branches were missed, we asked each LLM to write improved tests that would cover those gaps. Table 4 shows some of the coverage improvements for LLM 1.

Table 4: Branch Coverage Summary (LLM 1)

| Task | Base Coverage (%) | Improved Coverage (%) | Improvement |
|------|-------------------|-----------------------|-------------|
| #0   | 80                | 100                   | +20%        |
| #3   | 100               | 100                   | —           |
| #4   | 75                | 100                   | +25%        |
| #5   | 87                | 100                   | +13%        |
| #7   | 70                | 100                   | +30%        |
| #9   | 100               | 100                   | —           |
| #10  | 100               | 100                   | —           |
| #11  | 75                | 100                   | +25%        |
| #14  | 83                | 100                   | +17%        |
| #16  | 83                | 100                   | +17%        |
| #18  | 68                | 100                   | +32%        |
| #26  | 83                | 100                   | +17%        |
| #27  | 90                | 100                   | +10%        |
| #28  | 83                | 100                   | +17%        |
| #29  | 75                | 100                   | +25%        |
| #31  | 85                | 100                   | +15%        |
| #39  | 94                | 100                   | +6%         |

After the improvement step, **all 30 tasks reached 100% branch coverage for both LLMs** (see Figures 3 and 4). The coverage reports are stored under `report/images/`.

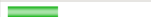
|                       |                                                                                   |      |                                                                                   |      |   |    |   |    |   |   |   |   |
|-----------------------|-----------------------------------------------------------------------------------|------|-----------------------------------------------------------------------------------|------|---|----|---|----|---|---|---|---|
| humaneval.llm1.task11 |  | %100 |  | %100 | 0 | 6  | 0 | 20 | 0 | 2 | 0 | 1 |
| humaneval.llm1.task39 |  | %100 |  | %100 | 0 | 12 | 0 | 18 | 0 | 3 | 0 | 1 |
| humaneval.llm1.task10 |  | %100 |  | %100 | 0 | 7  | 0 | 19 | 0 | 3 | 0 | 1 |
| humaneval.llm1.task4  |  | %100 |  | %100 | 0 | 6  | 0 | 13 | 0 | 2 | 0 | 1 |
| humaneval.llm1.task27 |  | %100 |  | %100 | 0 | 7  | 0 | 11 | 0 | 2 | 0 | 1 |
| humaneval.llm1.task25 |  | %100 |  | %100 | 0 | 6  | 0 | 12 | 0 | 2 | 0 | 1 |
| humaneval.llm1.task31 |  | %100 |  | %100 | 0 | 9  | 0 | 11 | 0 | 2 | 0 | 1 |
| humaneval.llm1.task0  |  | %100 |  | %100 | 0 | 7  | 0 | 9  | 0 | 2 | 0 | 1 |
| humaneval.llm1.task49 |  | %100 |  | %100 | 0 | 5  | 0 | 12 | 0 | 2 | 0 | 1 |
| humaneval.llm1.task18 |  | %100 |  | %100 | 0 | 10 | 0 | 15 | 0 | 2 | 0 | 1 |
| humaneval.llm1.task8  |  | %100 |  | %100 | 0 | 3  | 0 | 11 | 0 | 2 | 0 | 1 |
| humaneval.llm1.task5  |  | %100 |  | %100 | 0 | 6  | 0 | 9  | 0 | 2 | 0 | 1 |
| humaneval.llm1.task26 |  | %100 |  | %100 | 0 | 6  | 0 | 8  | 0 | 3 | 0 | 1 |
| humaneval.llm1.task16 |  | %100 |  | %100 | 0 | 5  | 0 | 7  | 0 | 2 | 0 | 1 |
| humaneval.llm1.task9  |  | %100 |  | %100 | 0 | 5  | 0 | 11 | 0 | 2 | 0 | 1 |
| humaneval.llm1.task7  |  | %100 |  | %100 | 0 | 7  | 0 | 9  | 0 | 2 | 0 | 1 |
| humaneval.llm1.task43 |  | %100 |  | %100 | 0 | 4  | 0 | 8  | 0 | 2 | 0 | 1 |
| humaneval.llm1.task28 |  | %100 |  | %100 | 0 | 5  | 0 | 8  | 0 | 2 | 0 | 1 |
| humaneval.llm1.task15 |  | %100 |  | %100 | 0 | 5  | 0 | 9  | 0 | 2 | 0 | 1 |
| humaneval.llm1.task14 |  | %100 |  | %100 | 0 | 5  | 0 | 7  | 0 | 2 | 0 | 1 |
| humaneval.llm1.task3  |  | %100 |  | %100 | 0 | 4  | 0 | 8  | 0 | 2 | 0 | 1 |
| humaneval.llm1.task29 |  | %100 |  | %100 | 0 | 5  | 0 | 4  | 0 | 3 | 0 | 1 |
| humaneval.llm1.task13 |  | %100 |  | %100 | 0 | 3  | 0 | 9  | 0 | 2 | 0 | 1 |
| humaneval.llm1.task30 |  | %100 |  | %100 | 0 | 4  | 0 | 4  | 0 | 3 | 0 | 1 |
| humaneval.llm1.task42 |  | %100 |                                                                                   | n/a  | 0 | 3  | 0 | 4  | 0 | 3 | 0 | 1 |
| humaneval.llm1.task34 |  | %100 |                                                                                   | n/a  | 0 | 2  | 0 | 2  | 0 | 2 | 0 | 1 |
| humaneval.llm1.task2  |  | %100 |                                                                                   | n/a  | 0 | 2  | 0 | 3  | 0 | 2 | 0 | 1 |
| humaneval.llm1.task45 |  | %100 |                                                                                   | n/a  | 0 | 2  | 0 | 2  | 0 | 2 | 0 | 1 |
| humaneval.llm1.task35 |  | %100 |                                                                                   | n/a  | 0 | 2  | 0 | 2  | 0 | 2 | 0 | 1 |
| humaneval.llm1.task23 |  | %100 |                                                                                   | n/a  | 0 | 2  | 0 | 2  | 0 | 2 | 0 | 1 |

Figure 3: JaCoCo Branch Coverage — LLM 1 Improved Tests (100% across all tasks)

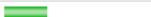
|                       |                                                                                     |      |                                                                                     |      |   |   |   |    |   |   |   |   |
|-----------------------|-------------------------------------------------------------------------------------|------|-------------------------------------------------------------------------------------|------|---|---|---|----|---|---|---|---|
| humaneval.llm2.task26 |  | %100 |  | %100 | 0 | 5 | 0 | 6  | 0 | 3 | 0 | 1 |
| humaneval.llm2.task4  |  | %100 |  | %100 | 0 | 4 | 0 | 11 | 0 | 2 | 0 | 1 |
| humaneval.llm2.task10 |  | %100 |  | %100 | 0 | 7 | 0 | 15 | 0 | 3 | 0 | 1 |
| humaneval.llm2.task39 |  | %100 |  | %100 | 0 | 8 | 0 | 17 | 0 | 3 | 0 | 1 |
| humaneval.llm2.task9  |  | %100 |  | %100 | 0 | 5 | 0 | 11 | 0 | 2 | 0 | 1 |
| humaneval.llm2.task5  |  | %100 |  | %100 | 0 | 4 | 0 | 9  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task0  |  | %100 |  | %100 | 0 | 5 | 0 | 7  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task8  |  | %100 |  | %100 | 0 | 3 | 0 | 8  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task25 |  | %100 |  | %100 | 0 | 5 | 0 | 11 | 0 | 2 | 0 | 1 |
| humaneval.llm2.task43 |  | %100 |  | %100 | 0 | 5 | 0 | 6  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task27 |  | %100 |  | %100 | 0 | 4 | 0 | 8  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task11 |  | %100 |  | %100 | 0 | 4 | 0 | 7  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task16 |  | %100 |  | %100 | 0 | 3 | 0 | 5  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task18 |  | %100 |  | %100 | 0 | 4 | 0 | 6  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task7  |  | %100 |  | %100 | 0 | 4 | 0 | 7  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task31 |  | %100 |  | %100 | 0 | 5 | 0 | 7  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task3  |  | %100 |  | %100 | 0 | 4 | 0 | 8  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task15 |  | %100 |  | %100 | 0 | 4 | 0 | 7  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task14 |  | %100 |  | %100 | 0 | 3 | 0 | 5  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task49 |  | %100 |  | %100 | 0 | 3 | 0 | 5  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task13 |  | %100 |  | %100 | 0 | 3 | 0 | 6  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task30 |  | %100 |  | %100 | 0 | 4 | 0 | 2  | 0 | 3 | 0 | 1 |
| humaneval.llm2.task42 |  | %100 |                                                                                     | n/a  | 0 | 3 | 0 | 2  | 0 | 3 | 0 | 1 |
| humaneval.llm2.task29 |  | %100 |                                                                                     | n/a  | 0 | 3 | 0 | 2  | 0 | 3 | 0 | 1 |
| humaneval.llm2.task34 |  | %100 |                                                                                     | n/a  | 0 | 2 | 0 | 4  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task45 |  | %100 |                                                                                     | n/a  | 0 | 2 | 0 | 2  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task35 |  | %100 |                                                                                     | n/a  | 0 | 2 | 0 | 2  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task2  |  | %100 |                                                                                     | n/a  | 0 | 2 | 0 | 2  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task28 |  | %100 |                                                                                     | n/a  | 0 | 2 | 0 | 2  | 0 | 2 | 0 | 1 |
| humaneval.llm2.task23 |  | %100 |                                                                                     | n/a  | 0 | 2 | 0 | 2  | 0 | 2 | 0 | 1 |

Figure 4: JaCoCo Branch Coverage — LLM 2 Improved Tests (100% across all tasks)

### 3.5 Manual Assessment Results

We wrote manual black-box tests for all 30 tasks on both LLMs. Each team member took 10 tasks and came up with equivalence classes (valid, invalid) and boundary conditions, then wrote JUnit 5 tests for each one. In total we ran 273 manual tests per LLM. The test files are under `src/test/java/humaneval/manual/` and the equivalence class tables are in `analysis/equivalence_classes/`.

Here are a few example equivalence class tables. The full set for all 30 tasks can be found in the repository.

Table 5: Equivalence Class Partitioning – Task #0 (hasCloseElements)

| Class ID                   | Type     | Description                                  | Expected           |
|----------------------------|----------|----------------------------------------------|--------------------|
| V1                         | Valid    | Pair closer than threshold                   | <code>true</code>  |
| V2                         | Valid    | No pair closer than threshold                | <code>false</code> |
| V3                         | Valid    | Identical elements with positive threshold   | <code>true</code>  |
| I1                         | Invalid  | Empty list (no pairs possible)               | <code>false</code> |
| I2                         | Invalid  | Single element list                          | <code>false</code> |
| <i>Boundary Conditions</i> |          |                                              |                    |
| BC1                        | Boundary | Distance exactly equals threshold (strict <) | <code>false</code> |
| BC3                        | Boundary | Threshold = 0 with distinct elements         | <code>false</code> |
| BC4                        | Boundary | Threshold = 0 with identical elements        | <code>false</code> |

Table 6: Equivalence Class Partitioning – Task #31 (isPrime)

| Class ID                   | Type     | Description                                      | Expected           |
|----------------------------|----------|--------------------------------------------------|--------------------|
| V1                         | Valid    | Small prime number (e.g., 3)                     | <code>true</code>  |
| V2                         | Valid    | Large prime number (e.g., 13441)                 | <code>true</code>  |
| V3                         | Valid    | Even composite > 2 (e.g., 6)                     | <code>false</code> |
| I1                         | Invalid  | Number 1 (not prime by definition)               | <code>false</code> |
| I2                         | Invalid  | Negative number (e.g., -7)                       | <code>false</code> |
| I3                         | Invalid  | Zero                                             | <code>false</code> |
| <i>Boundary Conditions</i> |          |                                                  |                    |
| BC1                        | Boundary | Smallest prime (2)                               | <code>true</code>  |
| BC4                        | Boundary | Perfect square (4, sqrt boundary)                | <code>false</code> |
| BC5                        | Boundary | Integer.MAX_VALUE (Mersenne prime $2^{31} - 1$ ) | <code>true</code>  |

Table 7: Equivalence Class Partitioning – Task #45 (triangleArea)

| Class ID                   | Type     | Description                            | Expected    |
|----------------------------|----------|----------------------------------------|-------------|
| V1                         | Valid    | Positive integers (5, 3)               | 7.5         |
| V2                         | Valid    | Positive floats (2.5, 4.2)             | 5.25        |
| I1                         | Invalid  | Negative side (−5, 3)                  | < 0         |
| I3                         | Invalid  | Side or height is zero (0, 5)          | 0.0         |
| <i>Boundary Conditions</i> |          |                                        |             |
| BC1                        | Boundary | Very small positive values (precision) | $\approx 0$ |
| BC2                        | Boundary | Double.MAX_VALUE overflow boundary     | MAX_VALUE   |
| BC3                        | Boundary | Double.MIN_VALUE (minimal positive)    | MIN_VALUE   |

### 3.6 Coverage Discussion and Input Mutations

Our manual tests covered all the equivalence classes and boundary conditions we identified. We ran 273 tests per LLM and the results are documented in `analysis/manual_test_reports/`.

**LLM 1:** 271 out of 273 tests passed (99.3%). The two failures were:

- Task #31 (`isPrime`): The loop condition `i * i <= n` causes integer overflow when testing `Integer.MAX_VALUE`.
- Task #45 (`triangleArea`): The multiplication `a * h` overflows to infinity when `a = Double.MAX_VALUE`.

**LLM 2:** 269 out of 273 tests passed (98.5%), with 3 failures and 1 timeout:

- Task #13 (`greatestCommonDivisor`): Does not handle negative inputs—returns negative GCD.
- Task #25 (`factorize`): The  $O(n)$  loop times out on a large prime (about 2.1 billion iterations).
- Task #45 (`triangleArea`): Same overflow problem as LLM 1.
- Task #49 (`modp`): Returns 1 instead of 0 when  $n = 0$  and  $p = 1$  because the initial value is not reduced by modulo.

This shows something important: even though both LLMs got 100% branch coverage, that was not enough to catch these bugs. The buggy code paths were technically “covered” but nobody tested them with extreme values. This matches what Foster et al. [2] found—49% of useful tests did not increase line coverage at all.

We did not need to create input mutations—the boundary values we picked by hand were enough to find all the bugs.

## 4 Results, Analysis, and Refactoring

### 4.1 Test Case Results

Table 8 shows how both LLMs did overall in the manual testing phase.

Table 8: Overall Test Results by LLM

| Metric               | LLM 1 (Gemini) | LLM 2 (Claude) |
|----------------------|----------------|----------------|
| Total Manual Tests   | 273            | 273            |
| Passed               | 271            | 269            |
| Failed               | 2              | 3              |
| Errors (Timeout)     | 0              | 1              |
| Pass Rate (%)        | 99.3           | 98.5           |
| Buggy Tasks          | 2/30           | 4/30           |
| Code Correctness (%) | 93.3           | 86.7           |

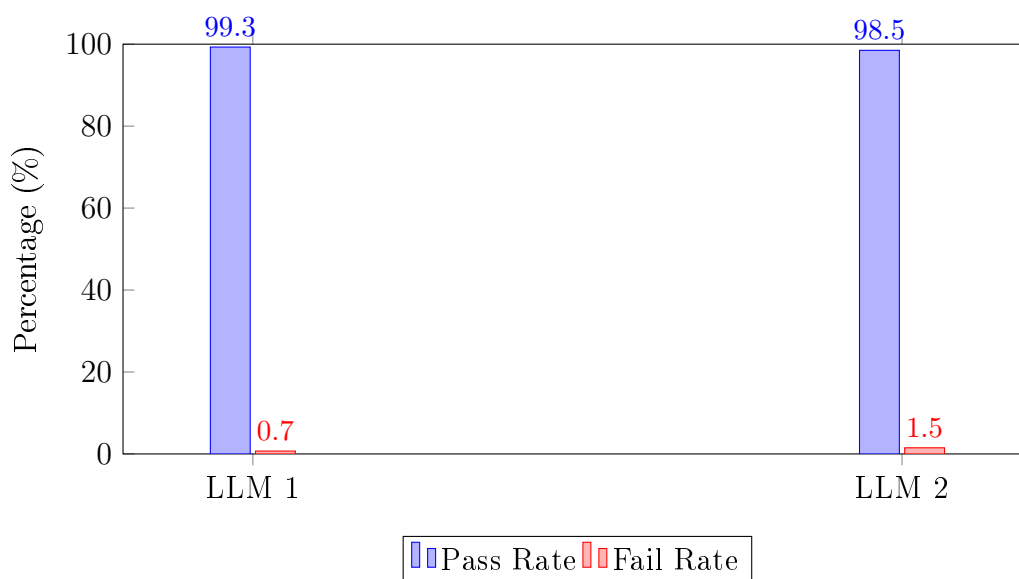


Figure 5: Test Pass/Fail Rates by LLM (Manual Black-Box Testing)

## 4.2 Statistical Evaluation

Both LLMs wrote code that works fine for normal inputs—all 30 tasks passed the base and improved tests with a 100% pass rate. The bugs only showed up when we ran the manual boundary tests. So both models are good at handling common cases but they have trouble with extreme inputs.

Some things worth noting:

- **Bug rate:** LLM 1 had bugs in 2 out of 30 tasks (6.7%), and LLM 2 had bugs in 4 out of 30 tasks (13.3%). So LLM 1 wrote more reliable code when it comes to edge cases.
- **Types of bugs:** LLM 1 only had overflow problems (integer and double). LLM 2 had a wider mix—missing input checks, a slow algorithm, and a wrong initial value.
- **Same bug in both:** Task #45 (`triangleArea`) was broken in both LLMs. They both wrote `a * h / 2.0` without thinking about what happens when `a` is very large.

- **Coverage is not enough:** All tasks had 100% branch coverage, but 6 bugs were still hiding in the code. This proves that coverage alone does not mean the code is correct.

### 4.3 LLM Comparison

Table 9 shows how the two LLMs compare task by task in terms of coverage and number of test methods.

Table 9: Per-Method Comparison Between LLMs (Improved Tests)

| Task           | LLM 1 Cov.(%) | LLM 1 Tests | LLM 2 Cov.(%) | LLM 2 Tests |
|----------------|---------------|-------------|---------------|-------------|
| #0             | 100           | 7           | 100           | 7           |
| #2             | 100           | 3           | 100           | 5           |
| #3             | 100           | 4           | 100           | 6           |
| #4             | 100           | 6           | 100           | 4           |
| #5             | 100           | 6           | 100           | 4           |
| #7             | 100           | 7           | 100           | 4           |
| #8             | 100           | 3           | 100           | 3           |
| #9             | 100           | 5           | 100           | 5           |
| #10            | 100           | 7           | 100           | 7           |
| #11            | 100           | 6           | 100           | 4           |
| #13            | 100           | 3           | 100           | 3           |
| #14            | 100           | 5           | 100           | 3           |
| #15            | 100           | 5           | 100           | 4           |
| #16            | 100           | 5           | 100           | 3           |
| #18            | 100           | 10          | 100           | 4           |
| #23            | 100           | 2           | 100           | 2           |
| #25            | 100           | 6           | 100           | 5           |
| #26            | 100           | 6           | 100           | 5           |
| #27            | 100           | 7           | 100           | 4           |
| #28            | 100           | 5           | 100           | 2           |
| #29            | 100           | 5           | 100           | 3           |
| #30            | 100           | 4           | 100           | 4           |
| #31            | 100           | 9           | 100           | 5           |
| #34            | 100           | 2           | 100           | 2           |
| #35            | 100           | 2           | 100           | 2           |
| #39            | 100           | 12          | 100           | 8           |
| #42            | 100           | 3           | 100           | 3           |
| #43            | 100           | 4           | 100           | 5           |
| #45            | 100           | 2           | 100           | 2           |
| #49            | 100           | 5           | 100           | 3           |
| <b>Average</b> | <b>100</b>    | <b>5.3</b>  | <b>100</b>    | <b>3.9</b>  |

LLM 1 (Gemini) wrote about 5.3 test methods per task on average, while LLM 2 (Claude) wrote about 3.9. Even with fewer tests, LLM 2 still got 100% coverage, so its tests were more focused. But LLM 2's actual code had more bugs, which shows that writing good tests and writing correct code are two different things.

## 4.4 Refactoring Process

Once we found the bugs through manual testing, we used the LLMs to fix them. The prompts and responses for each fix are saved in `llm_logs/llm1/refactoring/` and `llm_logs/llm2/refactoring/`. Here is how we did it:

1. We looked at the failing test to understand what went wrong and why.
2. We wrote a prompt that included the buggy code, the failing test, and what the correct behavior should be.
3. We gave the prompt to the LLM and got back the fixed code.
4. We applied the fix and re-ran all tests to make sure nothing else broke.

### LLM 1 fixes (2 bugs):

- Task #31 (`isPrime`): Changed `i * i <= n` to `(long) i * i <= n` to stop integer overflow.
- Task #45 (`triangleArea`): Changed `(a * h) / 2.0` to `a * (h / 2.0)` so the multiplication does not overflow first.

### LLM 2 fixes (4 bugs):

- Task #13 (`greatestCommonDivisor`): Added `Math.abs()` calls at the start to handle negative numbers.
- Task #25 (`factorize`): Changed the loop to only go up to  $\sqrt{n}$  instead of  $n$ .
- Task #45 (`triangleArea`): Same fix as LLM 1.
- Task #49 (`modp`): Changed `int result = 1` to `int result = 1 % p`.

## 4.5 Agent-Generated Code Errors

Table [10](#) lists all the bugs we found in the LLM-generated code.

Table 10: Errors Detected in LLM-Generated Code

| Task | LLM   | Error Type       | Description                                                                        | Resolution                   |
|------|-------|------------------|------------------------------------------------------------------------------------|------------------------------|
| #31  | LLM 1 | Integer Overflow | Loop condition <code>i*i&lt;=n</code> overflows for <code>Integer.MAX_VALUE</code> | Cast to <code>long</code>    |
| #45  | LLM 1 | Double Overflow  | Intermediate <code>a*h</code> overflows for <code>Double.MAX_VALUE</code>          | Reorder arithmetic           |
| #13  | LLM 2 | Missing Norm.    | No <code>Math.abs()</code> for negative GCD inputs                                 | Add <code>abs()</code> calls |
| #25  | LLM 2 | Performance      | $O(n)$ trial division too slow for large primes                                    | $O(\sqrt{n})$ optimization   |
| #45  | LLM 2 | Double Overflow  | Same intermediate overflow as LLM 1                                                | Reorder arithmetic           |
| #49  | LLM 2 | Off-by-One Init  | Initial value not reduced by modulo when $n = 0$                                   | Init as <code>1%p</code>     |

## 5 Conclusion

In this project we tested Gemini Pro (LLM 1) and Claude Opus 4.6 (LLM 2) on 30 Java tasks from HumanEval. Here is what we learned:

1. Both LLMs wrote code that works correctly for normal inputs. All 30 tasks passed the base tests without any changes to the source code.
2. After one round of test improvement, both LLMs got to 100% branch coverage on all 30 tasks. So they are good at generating tests when you tell them which branches to target.
3. But even with full coverage, manual testing found 2 bugs in LLM 1 and 4 bugs in LLM 2. All the bugs were in boundary conditions—overflow, missing input checks, slow algorithms, and wrong initial values. This confirms that **branch coverage by itself is not enough**.
4. LLM 1 (Gemini) had fewer bugs (6.7% vs. 13.3%), so it wrote more reliable code for edge cases. LLM 2 (Claude) wrote fewer but more focused tests.
5. Both LLMs were able to fix all the bugs when we gave them detailed prompts with the failing test and the root cause.
6. The LLM-generated tests had some test smells (mainly *Unknown Test* and *Magic Number Test*), which shows that LLMs tend to write quick-and-dirty tests rather than well-structured ones.

The bottom line is that LLMs are useful tools for writing code and tests, but you still need to do manual testing—especially boundary value analysis—to catch the subtle bugs that automated approaches miss.

## 6 Acknowledgments

### 6.1 GitHub Repository

The source code and all artifacts for this project are available at:

[https://github.com/cigdemahmet27/software\\_quality\\_and\\_testing\\_project](https://github.com/cigdemahmet27/software_quality_and_testing_project)

### 6.2 Work Distribution

Table 11: Work Distribution Among Team Members

| Member                               | Responsibilities                                                                                                                                                                                                                          |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Ahmet Enes Çiğdem – 150220079</b> | Project structure setup and prompt selection<br>LLM 1 (Gemini) code generation, base tests, and improved tests<br>Manual tests and equivalence classes for Tasks #3, #4, #5, #7, #9, #26, #27, #28, #29, #30<br>JNose test smell analysis |
| <b>Ali Eren Çiftçi – 150220022</b>   | LLM 2 (Claude) code generation and base tests<br>Manual tests and equivalence classes for Tasks #13, #23, #25, #31, #34, #35, #42, #43, #45, #49<br>Bug analysis, refactoring logs, and manual test reports                               |
| <b>Taha Çali – 150220050</b>         | LLM 2 (Claude) improved test generation<br>Manual tests and equivalence classes for Tasks #0, #2, #8, #10, #11, #14, #15, #16, #18, #39<br>JaCoCo coverage report generation<br>Literature review                                         |
| <b>Shared by all members</b>         | IEEE report writing                                                                                                                                                                                                                       |

## References

- [1] Y. Chen, Z. Hu, C. Zhi, J. Han, S. Deng, and J. Yin, “ChatUniTest: A framework for LLM-based test generation,” in *Companion Proc. 32nd ACM Int. Conf. Foundations of Software Engineering (FSE Companion ’24)*, Porto de Galinhas, Brazil, Jul. 2024, pp. 572–576. <https://doi.org/10.1145/3663529.3663801>
- [2] C. Foster, A. Gulati, M. Harman, I. Harper, K. Mao, J. Ritchey, H. Robert, and S. Sengupta, “Mutation-guided LLM-based test generation at Meta,” in *33rd ACM Int. Conf. Foundations of Software Engineering (FSE Companion ’25)*, Trondheim, Norway, Jun. 2025. <https://doi.org/10.1145/3696630.3728544>

- [3] J. Yoon, R. Feldt, and S. Yoo, “Adaptive testing for LLM-based applications: A diversity-based approach,” in *Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE ’25)*, 2025.
- [4] Q. Gu *et al.*, “LLM-based code generation method for Golang compiler testing,” in *Proc. 31st ACM Joint European Software Engineering Conf. and Symp. on the Foundations of Software Engineering (ESEC/FSE ’23)*, San Francisco, CA, USA, Dec. 2023.
- [5] S. Fakhoury, A. Naik, G. Sakkas, S. Chakraborty, and S. K. Lahiri, “LLM-based test-driven interactive code generation: User study and empirical evaluation,” *IEEE Trans. Softw. Eng.*, vol. 50, no. 9, pp. 2254–2267, Sep. 2024. <https://doi.org/10.1109/TSE.2024.3428972>
- [6] M. Chen *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.